University of Pretoria

Department of Computer Science

COS720 – Computer Information & Security I

ThreatSense

Hybrid AI-Powered Insider Threat Detection System

Technical Documentation

Student: Rhulani Matiane

Student No: u23875616

Date: 19 May 2026

Contents

1 System Overview

ThreatSense is a lightweight AI-powered prototype that detects potential insider threats by analysing employee behavioural activity records. Each record is classified as *Malicious* or *Normal*, assigned a Composite Risk Score (CRS), and accompanied by a plain-English explanation understandable by a non-technical manager.

The system implements a Hybrid Detection Engine combining two complementary models:

- **XGBoost**: supervised classifier trained on 94,891 labelled records
- **Isolation Forest**: unsupervised anomaly detector that flags statistically unusual behaviour

Their outputs are fused into the Composite Risk Score:

$$\text{CRS} = \min\big(100,\ (0.6\,P_{\text{XGB}} + 0.4\,S_{\text{IF}}) \times 100\big)$$

1.1 Project Structure

```
cos720-insider-threat/
|---- data/
|   \---- insider_threat_clean_dataset.csv
|---- models/
|   |---- xgboost.pkl
|   |---- random_forest.pkl
|   |---- isolation_forest.pkl
|   |---- encoders.pkl
|   |---- dept_stats.pkl
|   \---- threshold.pkl
|---- src/
|   |---- data/preprocess.py
|   |---- models/train.py
|   |---- models/predict.py
|   |---- explainability/explain.py
|   \---- utils/helpers.py
|   \---- utils/pdf_report.py
|---- prototype/
|   \---- app.py
|---- tests/
|   \---- test_scenarios.py
|---- docs/
|---- requirements.txt
\---- README.md
```

2 Software Requirements and Dependencies

2.1 Runtime Requirements

Requirement	Version	Notes
Python	3.9 – 3.13	Tested on 3.13 (Windows 11)
pip	$\geq$ 23.0	Included with Python
RAM	$\geq$ 4 GB	8 GB recommended for training
Disk	$\geq$ 500 MB	Dataset + model files
OS	Any	Windows, macOS, Linux

Table 1: Runtime environment requirements

2.2 Python Dependencies

Package	Min Version	Purpose
pandas	2.0.0	Data loading and manipulation
numpy	1.26.0	Numerical operations
scikit-learn	1.4.0	Isolation Forest, Random Forest, metrics
xgboost	2.0.0	Primary supervised classifier
imbalanced-learn	0.12.0	SMOTE (used in model comparison)
joblib	1.3.0	Model serialisation
streamlit	1.35.0	Web prototype framework
matplotlib	3.8.0	Charts and visualisations
seaborn	0.13.0	Confusion matrix heatmap
reportlab	4.0.0	PDF forensic report generation

Table 2: Python package dependencies

3 Installation Instructions

3.1 Step 1: Extract the ZIP File

Extract the submitted ZIP file to a location of your choice. The extracted folder should contain the full project structure as shown in Section 1.

3.2 Step 2: Create a Virtual Environment

Open a terminal in the extracted project folder and run:

```
# Windows (Command Prompt)
python -m venv venv
venv\Scripts\activate

# Windows (PowerShell)
python -m venv venv
venv\Scripts\Activate.ps1

# macOS / Linux
python -m venv venv
source venv/bin/activate
```

You should see `(venv)` at the start of your terminal prompt when active.

3.3 Step 3: Install Dependencies

```
pip install -r requirements.txt
```

4 Deployment and Execution Instructions

4.1 Step 1: Train the Models

```
python src/models/train.py
```

This will preprocess the dataset, train XGBoost, Random Forest, and Isolation Forest, run 5-fold cross-validation, evaluate on the test set, and save six `.pkl` model files to `models/`. Expected training time is 30–120 seconds depending on hardware.

4.2 Step 2: Run the Testing Scenarios

```
python tests/test_scenarios.py
```

This produces a full evaluation report including TP, FP, and FN scenario analysis, and saves confusion matrix plots to `docs/`.

4.3 Step 3: Launch the Prototype

```
streamlit run prototype/app.py
```

The prototype opens in your browser at `http://localhost:8501`. Press Enter to skip the email prompt on first launch.

4.4 Step 4: Generate a Batch Test File (Optional)

To test the Batch Processing page, run the following in Python:

```
import pandas as pd
df = pd.read_csv('data/insider_threat_clean_dataset.csv')
sample = pd.concat([
    df[df['is_malicious']==1].head(10),
    df[df['is_malicious']==0].head(10)
]).drop(columns=['is_malicious'])
sample.to_csv('data/batch_test_sample.csv', index=False)
```

Upload `data/batch_test_sample.csv` on the Batch Processing page.

5 Source Code Documentation

5.1 src/data/preprocess.py

Handles all data preprocessing for both training and inference.

- `preprocess(filepath)` — full pipeline returning X_train, X_test, y_train, y_test, encoders, dept_stats
- `encode_categoricals(df, encoders, fit)` — label-encodes 4 categorical columns
- `engineer_deviation_features(df, dept_stats, fit)` — computes z-score deviation from department mean for 4 behavioural features
- `preprocess_single_record(record, encoders, dept_stats)` — inference-time preprocessing for a single record dictionary

5.2 src/models/train.py

Trains all three models and saves artefacts to `models/`.

- `train_xgboost(X, y)` — trains XGBoost with `scale_pos_weight=17`
- `train_isolation_forest(X)` — trains Isolation Forest with `contamination=0.054`
- `train_random_forest(X, y)` — trains Random Forest for feature importance
- `cross_validate_model(xgb, X, y)` — 5-fold stratified cross-validation
- `evaluate(xgb, iso, X_test, y_test)` — full test set evaluation at threshold 0.65

Key hyperparameters: XGBoost — n_estimators=300, max_depth=6, learning_rate=0.1, scale_pos_weight=17. Isolation Forest — n_estimators=200, contamination=0.054. Classification threshold — 0.65 (F1-optimised).

5.3 src/models/predict.py

Handles inference for single records and batch DataFrames.

- `load_models()` — loads all 6 `.pkl` files from `models/`, returns (xgb, rf, iso, encoders, dept_stats, threshold)
- `predict_record(record, xgb, rf, iso, encoders, dept_stats, threshold)` — classifies one record, returns label, CRS, confidence, risk level, and IF anomaly flag
- `predict_dataframe(df, ...)` — batch prediction for all rows in a DataFrame

5.4 src/explainability/explain.py

Generates all explainability outputs.

- `get_global_feature_importance(rf, feature_names)` — global Random Forest feature importance ranking
- `get_local_explanation(rf, X, feature_names)` — per-record contribution scores
- `generate_forensic_report(local_df, result)` — ISO/IEC 27043-aligned three-phase text report (Readiness, Detection, Investigative)
- `get_risk_score_breakdown(local_df)` — top risk indicators for UI display

5.5 src/utils/pdf_report.py

Generates a downloadable PDF forensic investigation report using `reportlab`. The report adapts to the active UI theme (dark/light) via the `theme` parameter and includes all three ISO/IEC 27043 phases.

5.6 prototype/app.py

The Streamlit web prototype. Implements four pages:

Page	Features
Analyse Record	Sample or manual profile input; classification result; confidence gauge; AI explainability panel; PDF download
Batch Processing	CSV upload; bulk classification; risk distribution chart; results export
Model Overview	Performance metrics; feature importance chart; CV results; architecture table
Analysis History	Session log of all analyses with timestamp, score, and risk level

Table 3: Prototype pages and features

5.7 tests/test_scenarios.py

Runs the full evaluation pipeline at threshold 0.65 and produces a TP, FP, FN scenario report with individual end-to-end record tests and forensic reports.

6 Description of Trained Model Files

All model files are serialised using `joblib` and stored in `models/`. They are generated by running `python src/models/train.py`.

File	Size	Description
`xgboost.pkl`	~80 MB	Trained XGBoost classifier. Primary classification model.
`random_forest.pkl`	~100 MB	Trained Random Forest. Used only for feature importance in the explainability module.
`isolation_forest.pkl`	~15 MB	Trained Isolation Forest. Unsupervised anomaly detector.
`encoders.pkl`	<1 MB	Fitted LabelEncoder objects for 4 categorical columns.
`dept_stats.pkl`	<1 MB	Department mean and standard deviation statistics for deviation feature engineering.
`threshold.pkl`	<1 KB	Optimised classification threshold (0.65).

Table 4: Trained model artefacts

Model files are not included in the Git repository. Include the generated `.pkl` files in your ClickUP ZIP submission.

7 Demonstrating Prototype Functionality

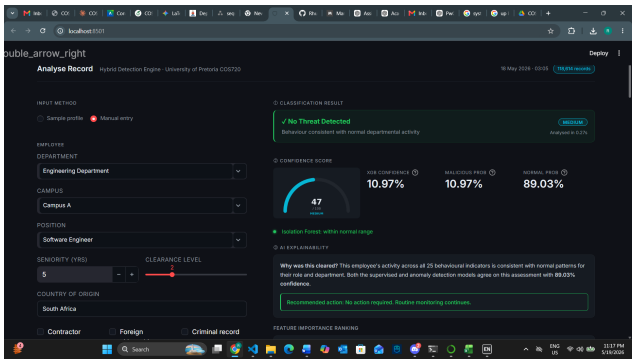

Figure 1: Analyse Record page

8 Troubleshooting Notes

Error	Solution
`ModuleNotFoundError: src`	Run all commands from the project root directory, not from inside a subfolder.
`FileNotFoundError: xgboost.pkl`	Run `python src/models/train.py` first to generate model files.
`FileNotFoundError: dataset.csv`	Download the dataset from Kaggle and place it in `data/`.
`ValueError: too many values to unpack`	Ensure all source files are updated. `load_models()` returns 6 values including the threshold.
`pip install` fails	Run `python -m pip install -upgrade pip` first, then retry.
HTML renders as raw text in prototype	Replace `app.py` with the latest version.
Black header bar in light mode	Hard refresh the browser with Ctrl+Shift+R.
Training takes too long	Reduce `n_estimators` to 100 in `train.py` for faster development iteration.

Table 5: Common issues and solutions

8.1 Verification Check

Run the following to verify the installation is working correctly:

```
import sys
sys.path.insert(0, '.')
from src.models.predict import load_models, predict_record
from src.utils.helpers import load_sample_profiles

xgb, rf, iso, encoders, dept_stats, threshold = load_models()
profiles = load_sample_profiles()
record = {k: v for k, v in profiles[1].items() if k != 'name'}
result = predict_record(record, xgb, rf, iso, encoders, dept_stats, threshold)
print(f"Label: {result['label']} | CRS: {result['composite_risk_score']}")
# Expected: Label: Malicious | CRS: 99.8
```